Fundamental Programming in JAVA

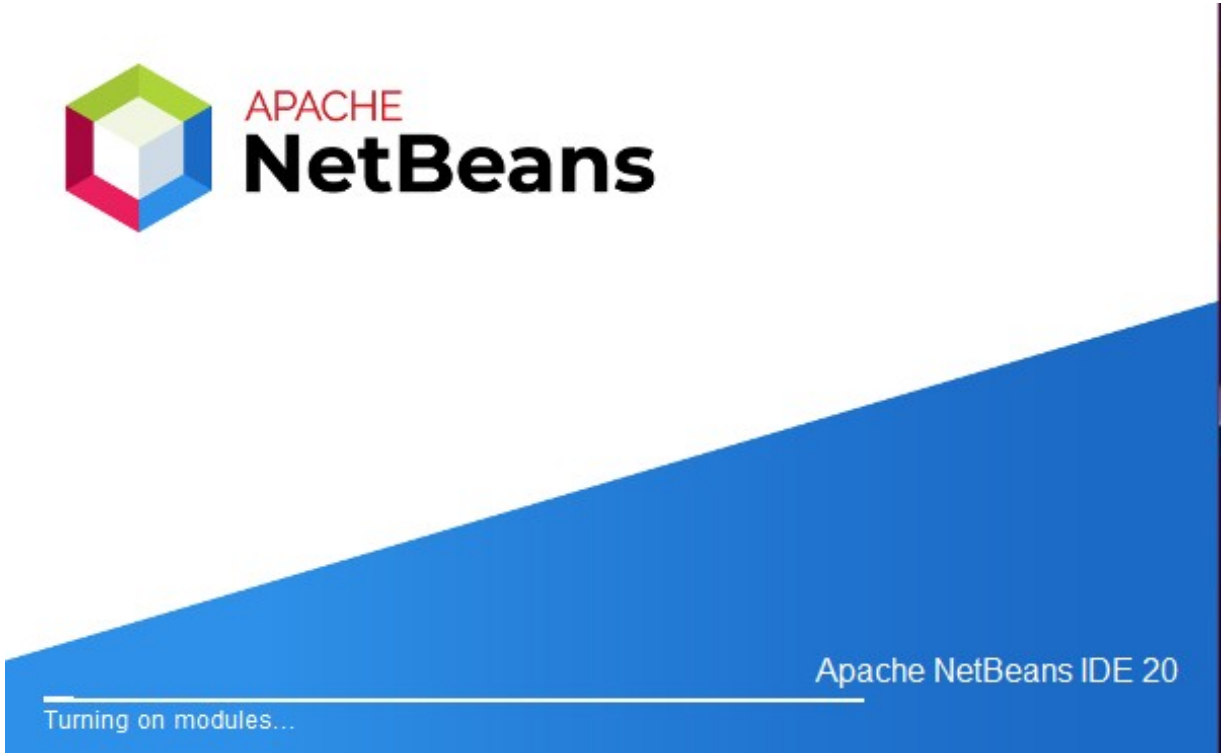
BASICS OF JAVA PROGRAMMING

Objectives

- Describe the structure of a simple Java program.
- Write a simple Java program.
- Be familiar with printing outputs in Java.
- Be aware of reserved keywords in Java.
- Use of comments in Java programs
- Become familiar with identifiers naming conventions in Java (e.g. variables , constants, classes , methods and packages).
- Become familiar with primitive data types
- To obtain input from the console using the Scanner class.
- To read a int,, float, double ,String ,char or Boolean value from the keyboard
- To cast the value of one type to another type.

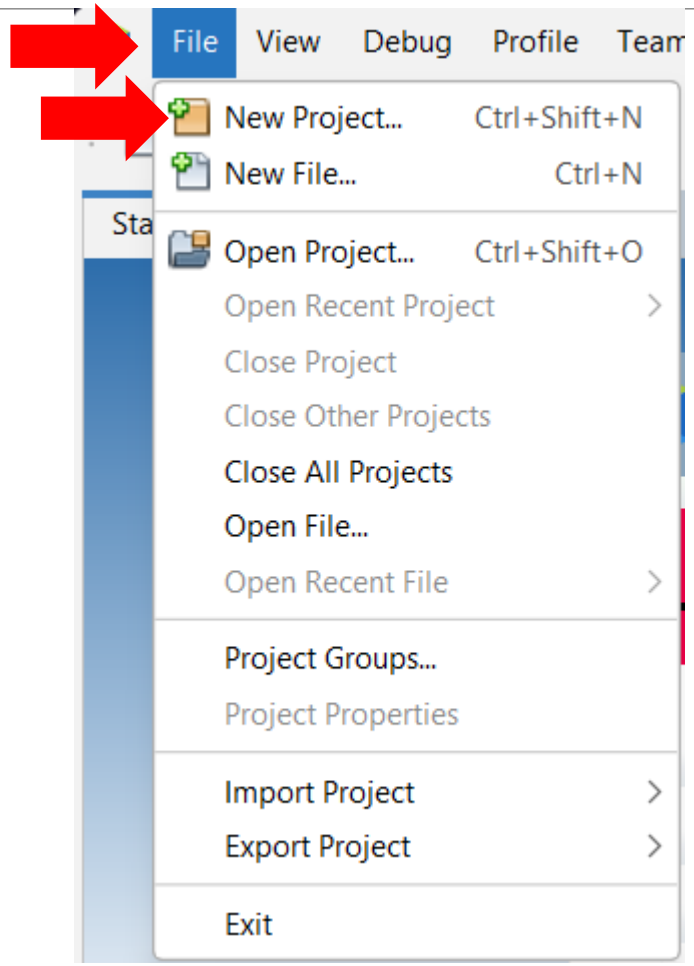
Creating first java project

- To create a Java project:
- Step one:
  - Start Apache NetBeans IDE.



Cont. creating first java project

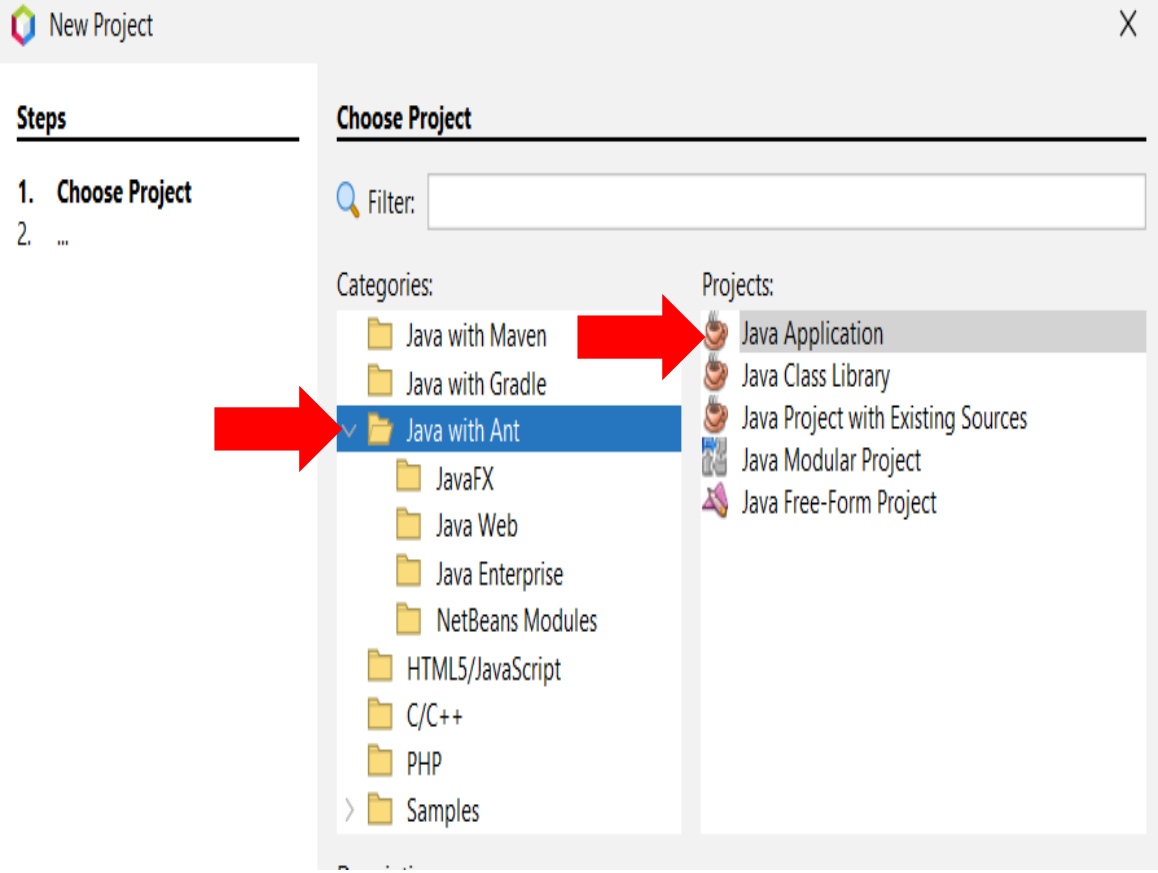
- Step two:
  - In the IDE, choose File → New Project or (Ctrl-Shift-N), as shown in the Figure:



Cont. creating first java project

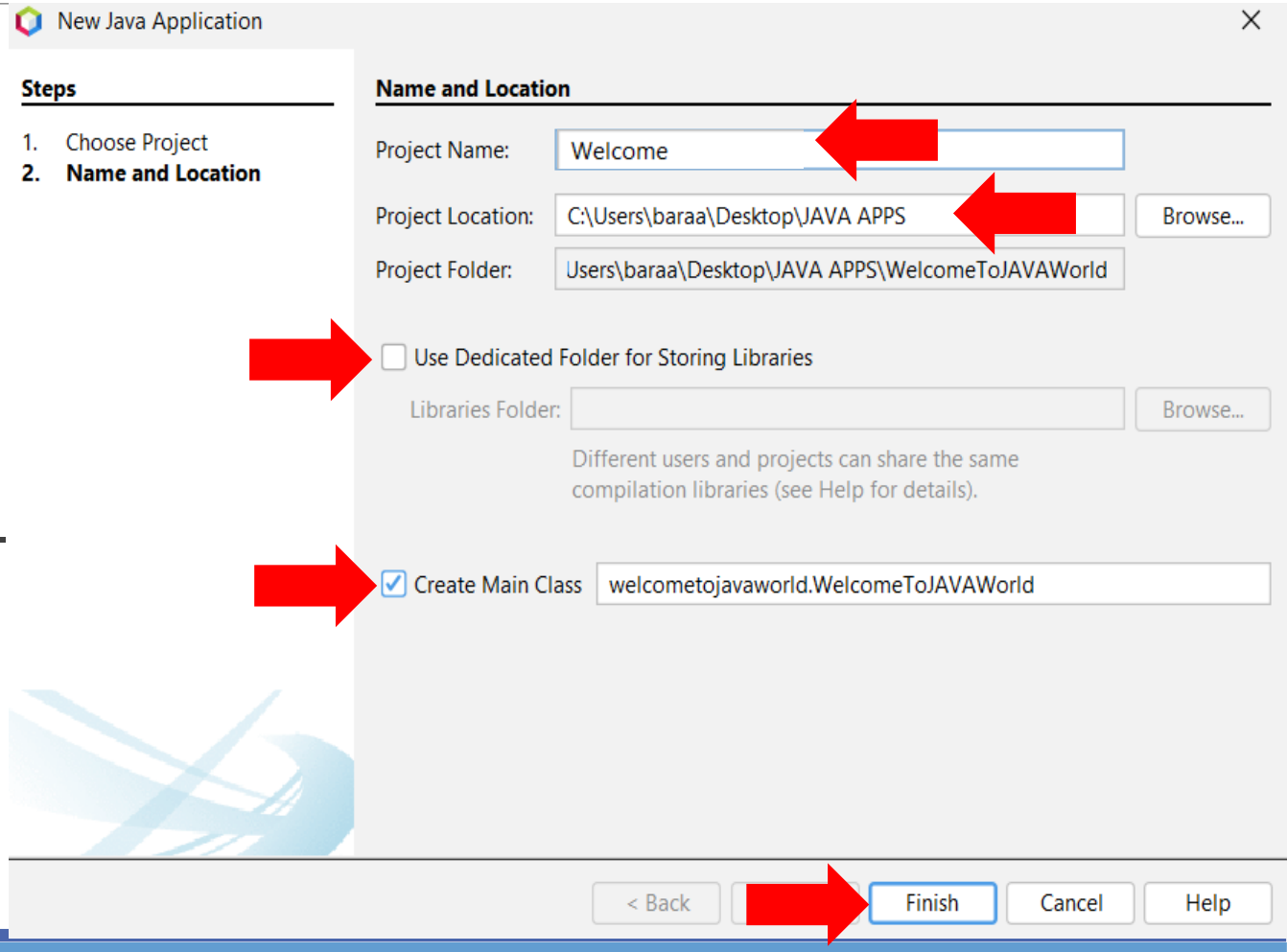
- Step three:
  - In the New Project wizard, from the Categories list choose Java with Ant.

→ from the Projects list choose Java Application
→ Click Next >.



Cont. creating first java project

- Step four:
  - In the Project Name text field, type: Welcome
  - Leave the Project Location text field to the default.
  - Leave the Use Dedicated Folder for Storing Libraries checkbox unchecked.
  - Leave the Create Main Class checkbox checked.
  - Click Finish.



Anatomy of Java Program

- A **block**: is a construct that groups of programming statements.
- Each block begins with an opening brace **{** and ends with a closing brace **}**.
- Every class has a **class block** that groups the data and methods of the class. Similarly, every method has a **method block** that groups the statements in the method. Blocks can be nested, meaning that one block can be placed within another.
- The following program contains two blocks (The class block and the main method block). The main method block is nested with the class block.

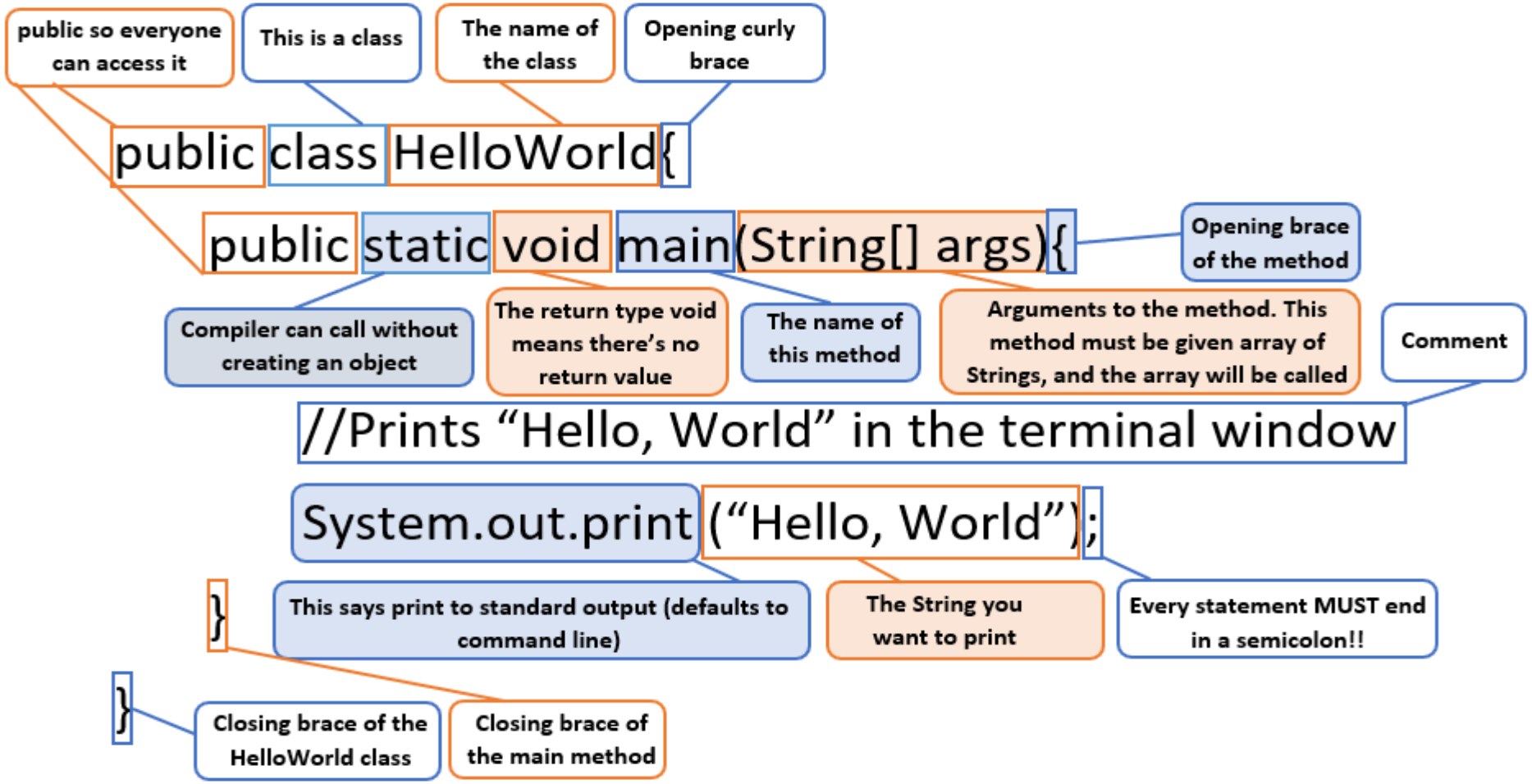
```
public class Welcome {
  public static void main(String[] args) {

  }
}
```

Class block

Method block

Anatomy of Java Program



Console Output (print, println)

Welcome.java

Source History

```java
public class Welcome {
  public static void main(String[] args) {
   int a = 100;
   System.out.print("a = ");
   System.out.println(a);

   System.out.print("a = " + a);
   System.out.print("\n");
   System.out.println(1.0/3.0);
   System.out.println("r = " + (1.0/3.0));
   System.out.print("\t Computer games");

}
}
```

String concatenation

Exercise

- Q: Write a Java code that gives the following outputs:
  - “Hello to the first Java program”
  - `10 + 23 = 33`

Identifiers

- Identifiers are programmer-defined names for:
  - Classes
  - Variables
  - Methods
- An identifier is a sequence of characters that consist of letters, digits, underscores (_), and dollar signs ($).
- An identifier must start with a letter, an underscore (_), or a dollar sign ($). It cannot start with a digit.

Identifiers

- An identifier cannot be a reserved word.
- An identifier cannot be **true**, **false**, or **null**.
- An identifier can be of any length.

Variables

- Variable: a memory location whose contents can be changed.

Memory allocation

Memory spaces after the statement length = 6.0; **executes**

Primitive Data Types

- Primitive data types are built into the Java language.
- There are 8 Java primitive data types.
  - *byte*
  - *short*
  - *int*
  - *long*
  - *float*
  - *double*
  - *boolean*
  - *char*

Numeric Data Types

byte	1 byte	**Integers** in the range **-128** to **+127**
short	2 bytes	**Integers** in the range of **-32,768** to **+32,767**
int	4 bytes	**Integers** in the range of **-2,147,483,648** to **+2,147,483,647**
long	8 bytes	**Integers** in the range of **-9,223,372,036,854,775,808** to **+9,223,372,036,854,775,807**
float	4 bytes	**Floating-point numbers** in the range of **±3.410-38** to **±3.41038**, with **7 digits of accuracy**
double	8 bytes	**Floating-point numbers** in the range of **±1.710-308** to **±1.710308**, with **15 digits of accuracy**

double vs. *float*

The *double* type values are more accurate than the *float* type values. For example,

```
System.out.println("1.0 / 3.0 is " + 1.0 / 3.0);
```

displays `1.0 / 3.0 is 0.3333333333333333`

16 digits

double is the default. Otherwise, you have to add letter **F** for *float* as the example follows:

```
System.out.println("1.0F / 3.0F is " + 1.0F / 3.0F);
```

displays `1.0F / 3.0F is 0.33333334`

7 digits

The *boolean* Data Type

- The Java *boolean* data type can have two possible values.
  - **true**
  - **false**
- The value of a *boolean* variable may only be copied into a *boolean* variable.

Java Program for *boolean* Variables

```java
public class BooleanExample {
    public static void main(String[] args) {
        boolean b1 = true;
        boolean b2 = false;
        boolean b3 = (b1 == b2);
        System.out.println(b1);
        System.out.println(b2);
        System.out.println(b3);
    }
}
```

Comparison Operator

The *char* Data Type

- The Java *char* data type provides access to single characters.
- *char* literals are enclosed in single quote marks.
  - ‘a’, ‘Z’, ‘\n’, ‘1’, ‘$’
- Don’t confuse *char* literals with *string* literals.
  - *char* literals are enclosed in **single quotes**.
  - *String* literals are enclosed in **double quotes**.

Java Program for *char* Variables

```
public class CharExample {
   public static void main(String[] args) {
      char char1='a';
      char char2='A’;
      char char3=65;

      System.out.println("char1: "+char1);
      System.out.println("char2: "+char2);
      System.out.println("char3: "+char3);
    }
}
```

Naming Conventions

- Choose meaningful and descriptive names.
- Class names:
  - Capitalize the first letter of each word in the name. For example, the class name
    - Eg. `ComputeExpression`.

Naming Conventions

- Method Name Identification
  - Capitalize the second and subsequent letters, without spaces between them,
    - Eg. (`getCustomerName(), setRentalItemPrice()`).
- Package name
  - Eg. (`java.util, java.lang`)

Naming Conventions

- Variables Identification:
  - Capitalize the second and subsequent letters, without spaces between them, e.g. (`customerName, customerCreditLimit`).
- Constants Identification (final variables):
  - All letters must be capitalized, e.g.(`MIN_WIDTH, MAX_NUMBER_OF_ITEMS`).
  - Can use underscore character (_) at the beginning or in the middle or at the end of the constant name.
  - Cannot use spaces between letters if the constant name is compound, e.g (`TAX RATE=0.16`);
- Uppercase and lowercase characters since the language is Case-Sensitive

Special Symbols

Character	Name	Description
{}	Opening and closing braces	Denotes a block to enclose statements.
()	Opening and closing parentheses	Used with methods.
[]	Opening and closing brackets	Denotes an array.
//	Double slashes	Precedes a comment line.
" "	Opening and closing quotation marks	Enclosing a string (i.e., sequence of characters).
;	Semicolon	Marks the end of a statement.

Constant Variable (final)

```
//This program computes the area of a circle
public class CircleArea {
    public static void main(String[] args) {
        double radius = 20.0;
        double area;
        //computer area
        final double PI = 3.14159;
        area = radius * radius * PI;
        //display area
        System.out.println("Circle area o radius " + radius + " is " + area);
    }
}
```

constant

Exercises

- Q: write a program to compute the net salary of an employee, suppose, tax rate = 7% and employee salary = 800.
  - tax amount = salary * tax rate.
  - net salary = salary - tax amount.
- Q: write a program to compute the Area of Rectangle.
  - width = 18.
  - length = 6.
- Q: write a program to Swap two float numbers using a temporary variable.
  - Number1 = 3.4.
  - Number2 = 73.22.

Programming Errors

- Syntax Errors
  - Detected by the compiler.

```
public class ShowSyntaxErrors {
  public static main(String[] args) {
    System.out.println("Welcome to Java);
  }
}
```

- Runtime Errors
  - Causes the program to abort.

```
public class ShowRuntimeErrors {
  public static void main(String[] args) {
    System.out.println(1 / 0);
  }
}
```

- Logic Errors
  - Produces incorrect result

```
public class ShowLogicErrors {
  public static void main(String[] args) {
    System.out.println("Celsius 35 is Fahrenheit degree
   ");
    System.out.println((9 / 5) * 35 + 32);
  }
}
```

Input Statement

- In Java, Scanner is a class in java.util package used for obtaining the input of the primitive types like int, double, etc. and strings.
- Using the Scanner class in Java is the easiest way to read input in a Java program, though not very efficient if you want an input method for scenarios where time is a constraint like in competitive programming.

How to Perform input using the Scanner class?

- Before using the Scanner class, it must be imported from the java.util package.
  import java.util.Scanner;
- In the statement that will perform the input, create an object (or instance) of the Scanner class.
  Scanner input;
- Specify the standard input stream System.in as the stream to get the data from.
  input = new Scanner(System.in);

Performing Input Statement

```
import java.util.Scanner;

public class Sample1 {
  public static void main(String[] args) {
   Scanner input = new Scanner(System.in);
   System.out.print("Enter integer number :");
   int num = input.nextInt();
   System.out.println("num = " + num);
   }}
```

1) import java package.

2) Declare identifier (such as: ***input, read, in, ...)*** and specify the input stream to get data from.

3) Use the declared scanner to input an integer value.

Input Statement

```
import java.util.Scanner;

public class Sample2 {
  public static void main(String[] args) {
   Scanner read = new Scanner(System.in);

   double d = read.nextDouble();
   System.out.println("You entered double " + d);

   byte b = read.nextByte();
   System.out.println("You entered byte " +
b);  }   }
```

Java Scanner Input Types

Method	Description
nextBoolean()	Used for reading Boolean value
nextByte()	Used for reading Byte value
nextDouble()	Used for reading Double value
nextFloat()	Used for reading Float value
nextInt()	Used for reading Int value
nextLine()	Used for reading Line value
nextLong()	Used for reading Long value
nextShort()	Used for reading Short value

How to Read *String* in Java
Next() Vs nextLine()

- The next() method of the Scanner class is used for getting input from the user. In order to use the next() method of the Scanner class, an instance of the Scanner class needs to be created. The next() method stops reading input from the user until it receives a white space(" "). So, it returns the next complete token from the Scanner.
- The nextLine() method of the Scanner class is also used for taking input from the user. In order to use the nextLine() method, we also need to create an instance of the Scanner class. The nextLine() method is capable of reading input till the end of the line. So, it stops reading input from the user when the user presses the enter key or line change.

How to Read Character in Java

- java Scanner class provides nextInt() method for reading an integer value, nextDouble() method for reading a double value, nextLong() method for reading a long value, etc. But there is no nextChar() method in the Scanner class to read a character in Java. In this slide, we will learn how to take character input in Java.
- To read a **character in Java**, we use next() of the **Scanner class method** followed by **chatAt(0)** at method of the String class.

How to Read Character in Java

```
import java.util.Scanner;
public class CharacterInputExample1{
public static void main(String[] args){
Scanner sc = new Scanner(System.in);
System.out.print("Input a character: ");
// reading a character
char c = sc.next().charAt(0);
//prints the character
System.out.println("You have entered "+c);
}}
```

Exercises

- Q: Write a java program that prompts the user to enter his name and displays the name using print statement .
- Q: Write a program that prompts a user to enter the width, and the height for a rectangle, computes the area and prints the outputs.
- Q :Read the employee's name, his total salary, calculate his net income after tax (10% of his salary) and print the result in a tabular form:

Mr, ……

Salary	tax	Net
---	---	---
1000	100	900

Exercises

- Q: Write a program that prompts the user to enter an integer. If the number is a multiple of 5, print HiFive. If the number is divisible by 2, print HiEven
- Q: Write program which will print a message “Pass” if a student mark value is between 50-100. And print a message “Fail” if student mark less than 50.

Data Conversion Rules

- When performing a binary operation involving two operands of different types, Java automatically converts the operand based on the following rules:
  - If one of the operands is *double*, the other is converted into *double*.
  - Otherwise, if one of the operands is *float*, the other is converted into *float*.
  - Otherwise, if one of the operands is *long*, the other is converted into *long*.
  - Otherwise, both operands are converted into *int*.

Data Conversion (Up Casting)

Consider the following statements:

```
byte i = 127;
System.out.println("i = " + i); // i = 127
double d = i * 3.1;
System.out.println("d = " + d); // d = 393.7
float a = i * 2.0F;
System.out.println("a = " + a); // a = 254.0
long k = 5 / 2L;
System.out.println("k = " + k); // k = 2
```

Type Casting (Down Casting)

- Two types of casting:
  - Widening Casting (automatically - Implicit casting) - converting a smaller type to a larger type size
    byte -> short -> char -> int -> long -> float -> double
  - Narrowing Casting (manually - Explicit casting) - converting a larger type to a smaller size type
    double -> float -> long -> int -> char -> short -> byte

Type Casting (Down Casting)

```
public class Demo {
  public static void main(String[] args) {
    double myDouble = 9.78;
    double d = 3; // automatic casting: int to double
    int myInt = (int) myDouble; // Manual casting: double to int
    System.out.println(myDouble);   // Outputs 9.78
    System.out.println(myInt); // Outputs 9  }
}
```

Casting in an Augmented Expression

- In Java, an augmented expression of the form **x1 op= x2** is implemented as **x1 = (T)(x1 op x2)**, where **T** is the type for **x1**.
- Therefore, the following code is correct.
- **int sum = 0;**
- **sum += 4.5**; // sum becomes 4 after this statement
- **sum += 4.5** is equivalent to **sum = (int)(sum + 4.5).**

Parsing Numeric String

- A string consisting of only integers or decimal numbers is called a **numeric string**
  - To convert a ***String*** consisting of an **integer to a value of the type *int***, we use the following expression:
  - Integer.parseInt(strExpression)
    - Integer.parseInt("6723") = 6723
    - Integer.parseInt("-823") = -823

Parsing Numeric String

- To convert a string consisting of a decimal number to a value of the type float or double type, we use the following expressions:
  - Float.parseFloat(strExpression)
  - Double.parseDouble(strExpression)
    - Float.parseFloat("34.56") = 34.56
    - Float.parseFloat("-542.97") = -542.97
    - Double.parseDouble("345.78") = 345.78
    - Double.parseDouble("-782.873") = -782.873

References

- Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019.
- lecture Notes: edugate.asu.edu.jo.
- lecture Videos: YouTube channel.
- Lab manual 1, 2.